

## REINFORCEMENT LEARNING LABORATORY

### Lab Assignment 4

*MATLAB Reinforcement Learning Onramp: Agent Training and Learning Curve Analysis*

|      |               |              |                  |
|------|---------------|--------------|------------------|
| Name | Sanju Bhaskar | Register No. | BL.EN.R4AIE26007 |
|------|---------------|--------------|------------------|

## REINFORCEMENT LEARNING ONRAMP

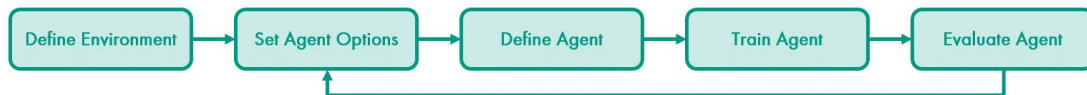


Figure 1: The reinforcement learning workflow followed in the Onramp — define environment, set agent options, define agent, train agent, evaluate agent.

### 1. Task 1 — Completion of the MATLAB RL Onramp

All modules of the Reinforcement Learning Onramp were completed, including the interactive exercises, the executable MATLAB examples and the end-of-chapter assessments. The chapter-wise record is given below.

Learn the basics of creating intelligent controllers that learn from experience in MATLAB®. Add a reinforcement learning agent to a Simulink® model and use MATLAB to train it to choose the best action in a given situation.

#### Course modules

✓ > [Overview of Reinforcement Learning](#) 100% | 20 min

✓ > [Define the Environment](#) 100% | 0.5 hours

✓ > [Define Agents](#) 100% | 1 hour

✓ > [Train Agents](#) 100% | 25 min

✓ > [Conclusion](#) 100% | 10 min

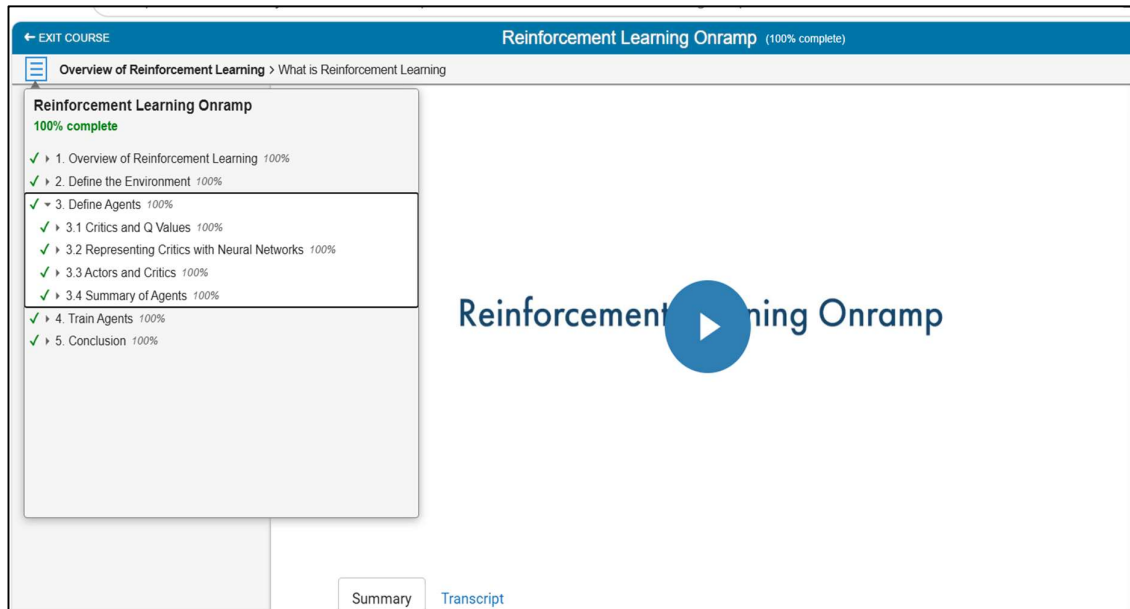
| # | Chapter / Module                | Key Topics Covered                                                     |
|---|---------------------------------|------------------------------------------------------------------------|
| 1 | What Is Reinforcement Learning? | Agent, environment, state, action, reward; RL vs. supervised learning  |
| 2 | The Dice Game                   | Episodic discrete environment; observations, actions and reward signal |

|   |                                           |                                                                                          |
|---|-------------------------------------------|------------------------------------------------------------------------------------------|
| 3 | Define Environments                       | rlFiniteSetSpec, rlNumericSpec, step and reset functions, rlFunctionEnv                  |
| 4 | Define Agents                             | rlTable and rlQAgent, critics, rlDQNAgent, rlPPOAgent, actor–critic structures           |
| 5 | Representing Critics with Neural Networks | dlnetwork, featureInputLayer, fullyConnectedLayer, reluLayer; Q-value functions          |
| 6 | The Warehouse Robot                       | Continuous observation and action spaces; DDPG actor and critic                          |
| 7 | Train Agents / Changing Options           | rlOptimizerOptions, rlDQNAgentOptions, $\epsilon$ -greedy exploration, rlTrainingOptions |
| 8 | Training Results and Further Practice     | Episode Manager, learning curves, convergence, simulating the trained policy             |

### 1.1 Course Completion Certificate



### 1.2 Course Progress Screen



## 2. Task 2 — Training an RL Agent in MATLAB

### 2.1 Step 1 — Create the Environment

The Onramp exercise uses the dice game environment, referred to in the Episode Manager as `targetDice`. The observation is the current target value, a finite set of 21 possible states (0 to 20), and the action is the choice of one of five dice, a finite set of five elements. A reward of  $-1$  is issued for every roll, so the return of an episode is the negative of the number of rolls taken; maximising reward is therefore equivalent to reaching the target in as few rolls as possible.

```
% Observation and action specifications
obsInfo = rlFiniteSetSpec(0:20);      % 21 possible targets
actInfo = rlFiniteSetSpec(1:5);      % 5 dice to choose from

% Environment built from custom step and reset functions
env = rlFunctionEnv(obsInfo, actInfo, @diceStep, @diceReset);
```

### 2.2 Step 2 — Create the Agent

Because the dice game has only 21 states and 5 actions, it can be represented exactly by a Q-table, so the agent trained in the Onramp training exercise is a tabular Q-learning agent, `rlQAgent`, backed by a  $21 \times 5$  `rlTable`. A Deep Q-Network agent, `rlDQNAgent`, was subsequently trained on the same environment with the table replaced by a vector Q-value function critic, so that the two can be compared.

```
T = rlTable(obsInfo, actInfo);        % rlTable: Table: [21x5 double]
critic = rlQValueFunction(T, obsInfo, actInfo);
agent = rlQAgent(critic, agentOpts);  % tabular Q-learning agent

% Neural-network alternative used in the second run
critic = rlVectorQValueFunction(net, obsInfo, actInfo);
agent = rlDQNAgent(critic, agopts);   % rlDQNAgent
```

The critic network used by the DQN agent takes one input feature (the observation) and produces five outputs, one Q-value per die:

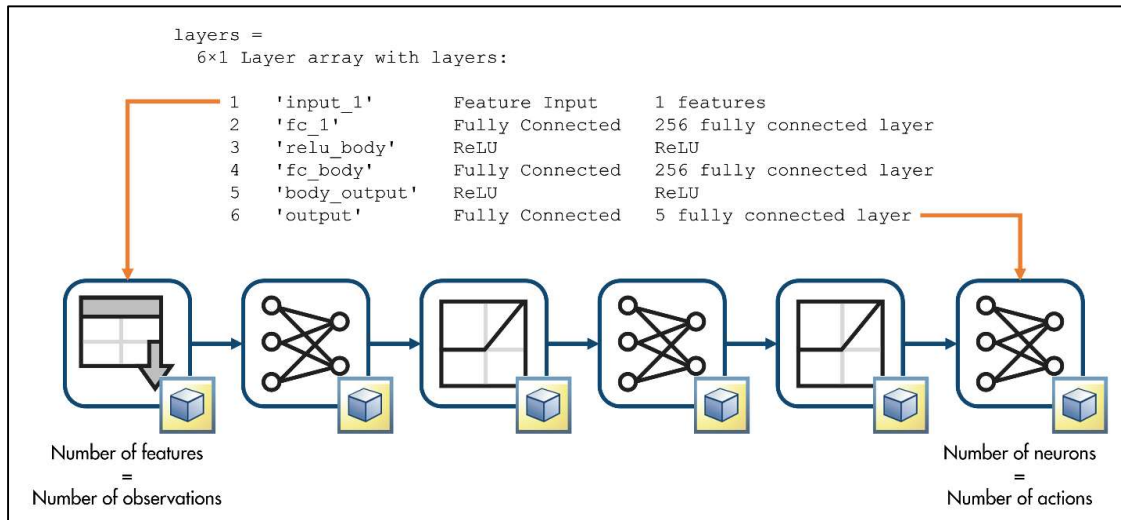


Figure 4: Layer array and schematic of the Q-value critic network — one input feature, two 256-unit fully connected layers with ReLU activations, and a five-unit output corresponding to the five dice.

## 2.3 Step 3 — Configure the Training Options

```
criticopts = rlOptimizerOptions( ...
    LearnRate = 1e-3, GradientThreshold = 1);

agopts = rlDQNAgentOptions( ...
    SampleTime = 1, DiscountFactor = 1, ...
    ExperienceBufferLength = 10000, MiniBatchSize = 64, ...
    UseDoubleDQN = 1, TargetSmoothFactor = 1e-3, ...
    CriticOptimizerOptions = criticopts);

trainopts = rlTrainingOptions( ...
    MaxEpisodes = 2000, ...
    StopTrainingCriteria = "AverageReward", StopTrainingValue = -3);
```

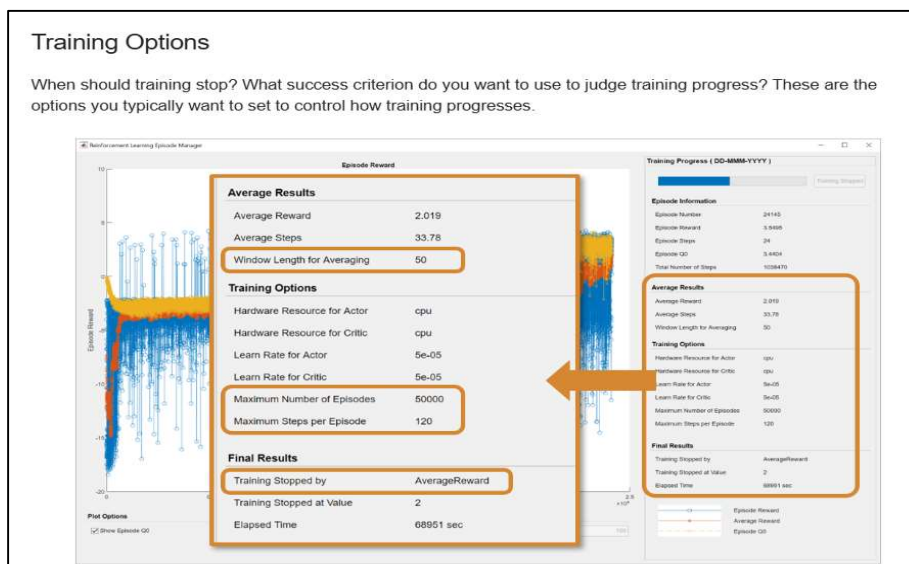


Figure 5: Training options and the corresponding fields of the Episode Manager — averaging window, stopping criterion and final results.

## 2.4 Step 4 — Train the Agent and Observe Progress

```
trainStats = train(agent, env, trainopts);
```

Training was monitored in the Episode Manager. The panel below was captured partway through the run, at episode 377 of 2000, when the average reward was still around  $-3.49$  and the status was still “Training”.

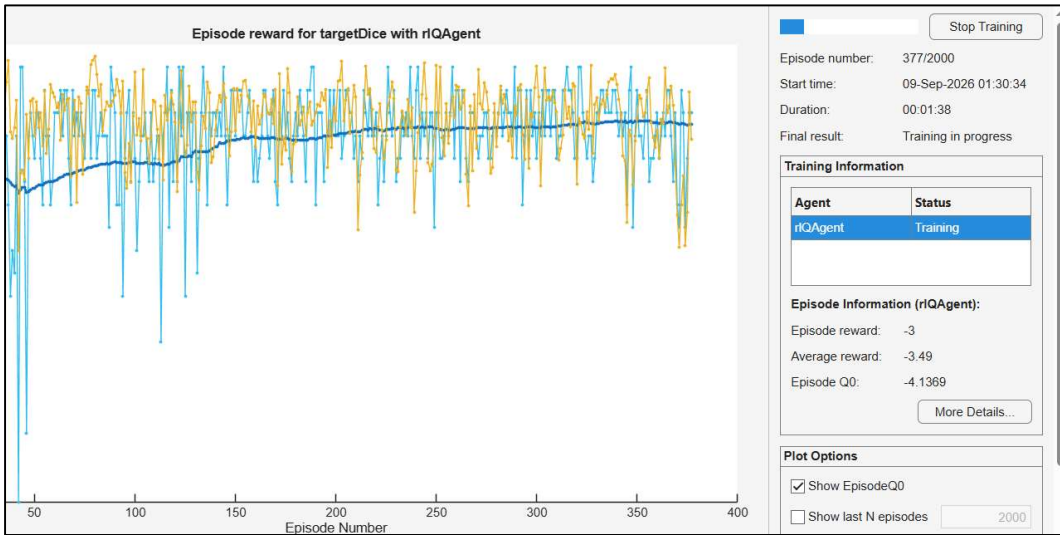
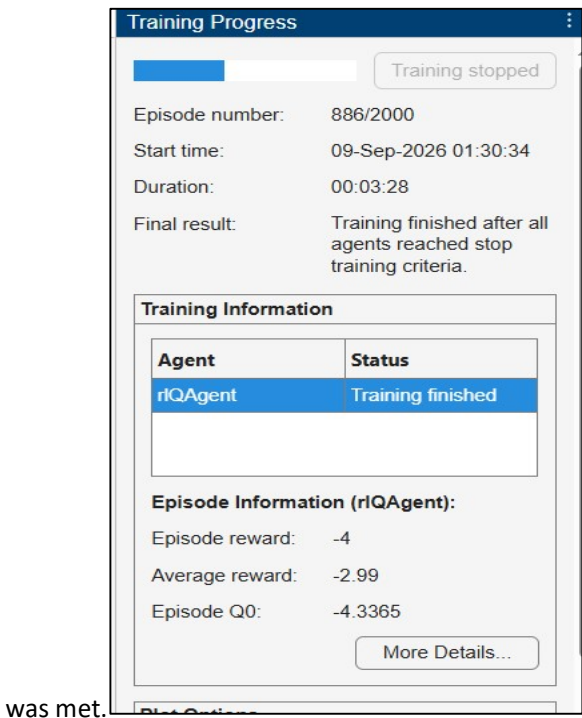


Figure 6: RL agent training in progress (episode 377/2000, duration 00:01:38, average reward  $-3.49$ ).

The run terminated on its own at episode 886, because the stopping criterion on average reward



was met.

Figure 7: Final training progress panel — training finished at episode 886 of 2000 after the agent reached the stop-training criteria.

## 2.5 Observation Table

| Parameter                              | Observation                                                                                                                                                                       |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>RL Environment</b>                  | targetDice — the dice game: an episodic, discrete, custom MATLAB environment (rlFunctionEnv) with 21 observations (target 0–20) and 5 actions (dice 1–5); reward of $-1$ per roll |
| <b>Agent Used</b>                      | rlQAgent — tabular Q-learning agent with a $21 \times 5$ rlTable critic and $\epsilon$ -greedy exploration                                                                        |
| <b>Number of Training Episodes</b>     | 886 episodes completed out of a maximum of 2000 (MaxEpisodes = 2000)                                                                                                              |
| <b>Initial Performance</b>             | Average reward began near $-4$ and fell to about $-6.4$ by episode 40; individual episodes reached below $-20$ (i.e. more than 20 rolls to reach the target)                      |
| <b>Final Performance</b>               | Average reward $-2.99$ ; final episode reward $-4$ ; Episode Q0 $-4.3365$ — approximately 3 rolls per episode                                                                     |
| <b>Training Time</b>                   | 00:03:28 (started 09-Sep-2026 01:30:34)                                                                                                                                           |
| <b>Training Stopped / Converged At</b> | Episode 886 — “Training finished after all agents reached stop training criteria” (AverageReward criterion satisfied)                                                             |

## 2.6 Behaviour of the Trained Agent

The Q-values of the trained agent were visualised across all 21 observations and 5 actions. The crosses mark the greedy action selected for each target value; the learned policy chooses low-sided dice for small targets and progressively higher-sided dice as the target increases, which is exactly the behaviour required to finish the game quickly without overshooting.

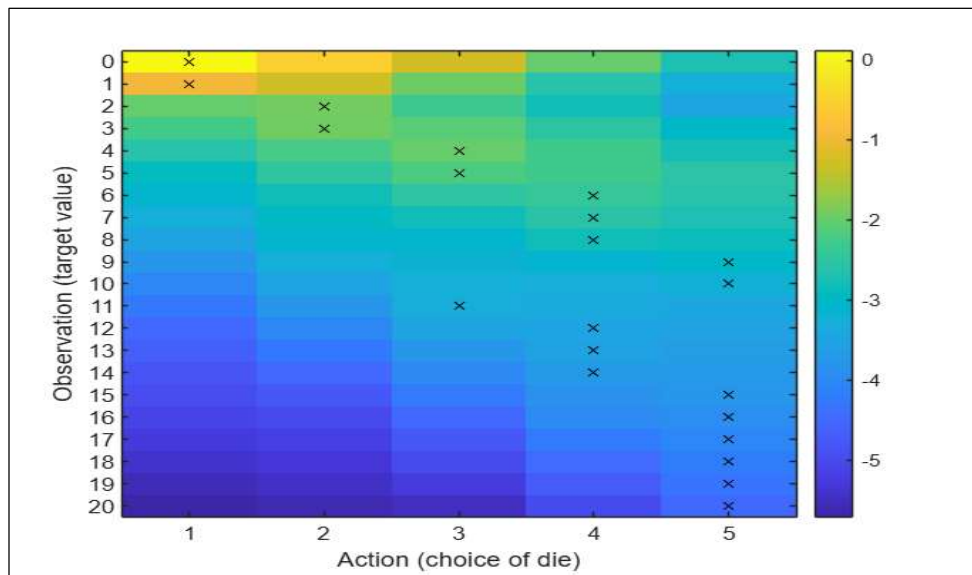


Figure 8: Learned Q-values and greedy policy of the trained rlQAgent — observation (target value) against action (choice of die).

Simulating the trained policy on fresh episodes gave the following console output; the target was consistently reached in a small number of rolls.

| Target | Die | Sides       | Roll |
|--------|-----|-------------|------|
| 17     | 5   | 3 4 5 7 8 8 | 8    |
| 9      | 5   | 3 4 5 7 8 8 | 4    |
| 5      | 3   | 1 4 4 4 5 7 | 4    |
| 1      | 1   | 1 1 1 5 5 8 | 1    |
| 0      | -   | - - - - - - | -    |

Target reached in 4 steps

Steps taken over five simulated episodes: 4, 4, 3, 5, 2

## 2.7 Questions

### Q1. What type of RL environment is used in the Onramp exercise?

The main exercise uses the dice game (targetDice): a custom, episodic, fully discrete MATLAB environment built with `rlFunctionEnv` from user-written step and reset functions. Its observation space is a finite set of 21 target values (0–20) and its action space is a finite set of five dice, both declared using `rlFiniteSetSpec`. The environment is stochastic, because the face obtained on a chosen die is random. A second environment, the warehouse robot, is also introduced; it is continuous, with observations and actions declared using `rlNumericSpec`.

### Q2. What type of agent is trained?

The agent trained in the Onramp training exercise is `rlQAgent`, a tabular Q-learning agent whose critic is a 21×5 `rlTable` — appropriate here because the state and action spaces are small enough to be enumerated exactly. A Deep Q-Network agent (`rlDQNAgent`) with an `rlVectorQValueFunction` critic over a six-layer `dlnetwork` was also created and trained on the same environment, and a PPO actor–critic agent (`rlPPOAgent`) is demonstrated. For the continuous warehouse-robot problem a DDPG agent is used, with an `rlContinuousDeterministicActor` and an `rlQValueFunction` critic.

### Q3. What is the purpose of training the agent?

Training adjusts the entries of the Q-table (or, for DQN, the weights of the critic network) so that the estimated Q-values approach the true expected return for each state–action pair. Once these estimates are accurate, acting greedily with respect to them yields a policy that maximises expected cumulative reward. Since the dice game gives –1 per roll, maximising return is exactly equivalent to learning which die to choose at each target so that the game finishes in the fewest possible rolls — a policy the agent discovers purely from interaction, without being told the rules of the game.

### Q4. What performance measure is used during training?

The Episode Manager reports three quantities per episode: the episode reward (total reward accumulated in that episode, shown in light blue), the average reward (a moving average of the episode reward over the averaging window, shown in dark blue), and Episode Q0 (the critic's predicted return from the initial state, shown in orange). The moving average is the primary measure of progress and was used here as the stopping criterion, `StopTrainingCriteria = "AverageReward"`; Q0 serves as a check on the accuracy of the critic.

**Q5. How does the agent's performance change as training progresses?**

In this run the average reward started near  $-4$ , dipped to about  $-6.4$  around episode 40 while  $\epsilon$  was still high and the Q-table was largely unfilled, then climbed steadily to roughly  $-3.4$  by episode 330 and settled in a band between about  $-3.0$  and  $-3.9$  for the remainder of training, finishing at  $-2.99$ . Individual episode rewards fell below  $-20$  in the first 50 episodes but were confined to roughly  $-1$  to  $-8$  after episode 150, so both the mean and the spread improved. The final Episode Q0 of  $-4.3365$  is close to the observed returns, indicating the value estimates had become reasonable.

**3. Task 3 — Learning Curve Analysis**

**3.1 Learning Curve**

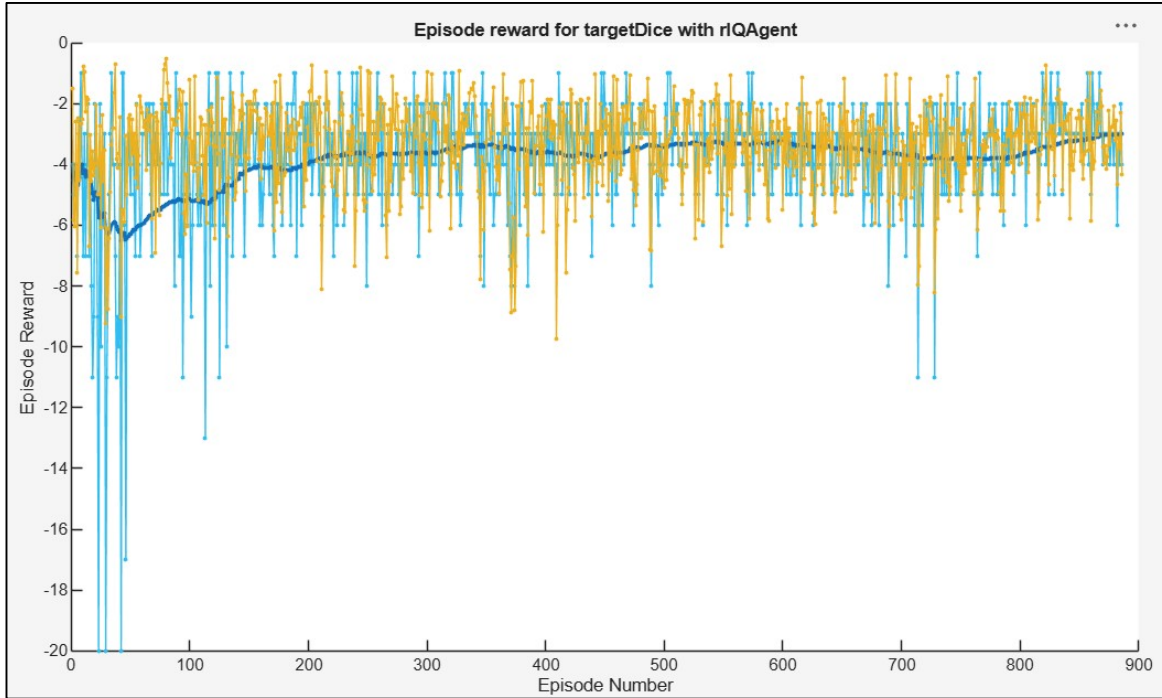


Figure 9: Learning curve — episode reward (light blue), moving average reward (dark blue) and Episode Q0 (orange) against episode number, for targetDice with rlQAgent.

**3.2 Stage-wise Analysis**

| Training Stage | Episode Range | Reward / Performance                                  | Observation                                                                                                                                                                                                      |
|----------------|---------------|-------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Initial        | 1 – 50        | Average $-4$ falling to $-6.4$ ; episodes below $-20$ | Near-random behaviour with $\epsilon$ at its maximum and the Q-table essentially empty. The average actually gets worse before it improves, as early optimistic estimates are corrected. Extremely wide scatter. |

|                        |           |                                                            |                                                                                                                                                                        |
|------------------------|-----------|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Early Training</b>  | 50 – 150  | Average rises from –6.4 to about –4.5                      | First clear upward trend as $\epsilon$ decays and useful Q-values accumulate. Worst-case episodes improve from below –20 to about –13.                                 |
| <b>Middle Training</b> | 150 – 350 | Average rises from –4.5 to about –3.4                      | Steepest sustained improvement. Episode Q0 begins to track the actual returns and the scatter narrows markedly, with almost no episode worse than –10.                 |
| <b>Late Training</b>   | 350 – 700 | Average oscillates between about –3.4 and –3.9             | Improvement largely stops and the curve flattens. Small oscillations remain, driven by the randomness of the dice rather than by continued learning.                   |
| <b>Final</b>           | 700 – 886 | Average reaches –2.99; final episode reward –4; Q0 –4.3365 | Average reward stable within a narrow band; the stopping criterion is satisfied and training halts automatically at episode 886, well short of the 2000-episode limit. |

| Summary Metric                  | Value                                                                                                                |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Initial average reward          | $\approx -4$ (worst point $\approx -6.4$ near episode 40)                                                            |
| Best episode reward observed    | $\approx -1$ (target reached in a single roll)                                                                       |
| Worst episode reward observed   | Below –20, within the first 50 episodes                                                                              |
| Final average reward            | –2.99                                                                                                                |
| Overall improvement             | $\approx 3.4$ reward units from the worst point ( $-6.4 \rightarrow -2.99$ ), i.e. roughly 3 fewer rolls per episode |
| Approximate convergence episode | $\approx 350$ (curve flat thereafter); training formally stopped at 886                                              |

### 3.3 Comparison with the DQN Agent

The same environment was also trained with a Deep Q-Network agent. It reached the stopping criterion at episode 425 in 00:02:18, with a final average reward of –3, an episode reward of –2 and Episode Q0 of –2.3017 — a similar final policy reached in fewer episodes, at the cost of a much larger set of parameters than the 105-entry Q-table.

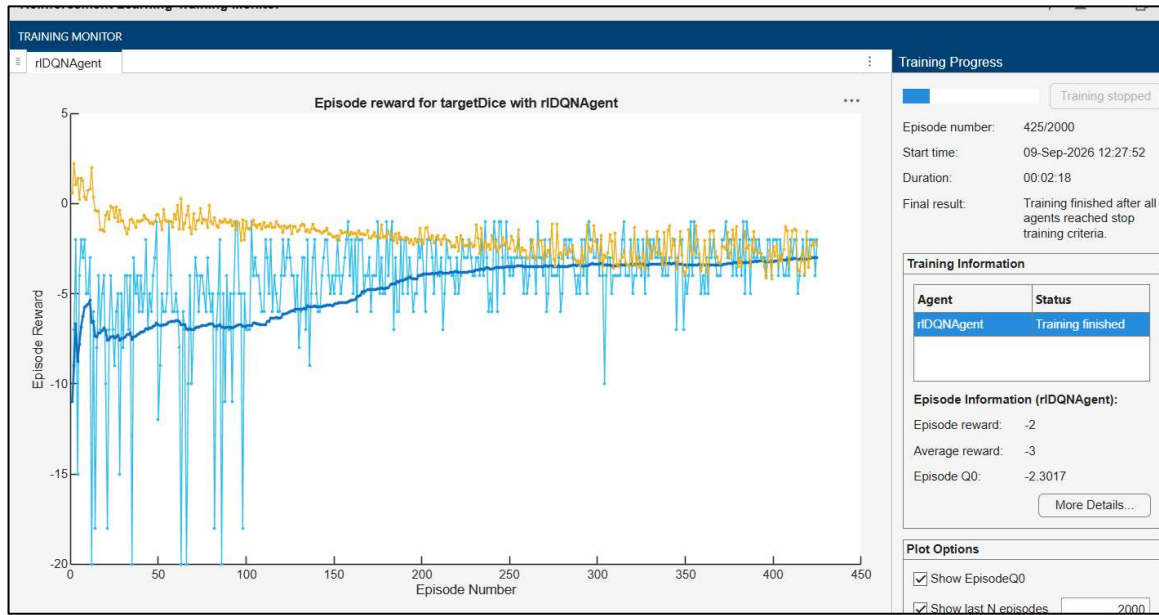


Figure 10: Training monitor for targetDice with rIQNAgent — training finished at episode 425 of 2000, duration 00:02:18.

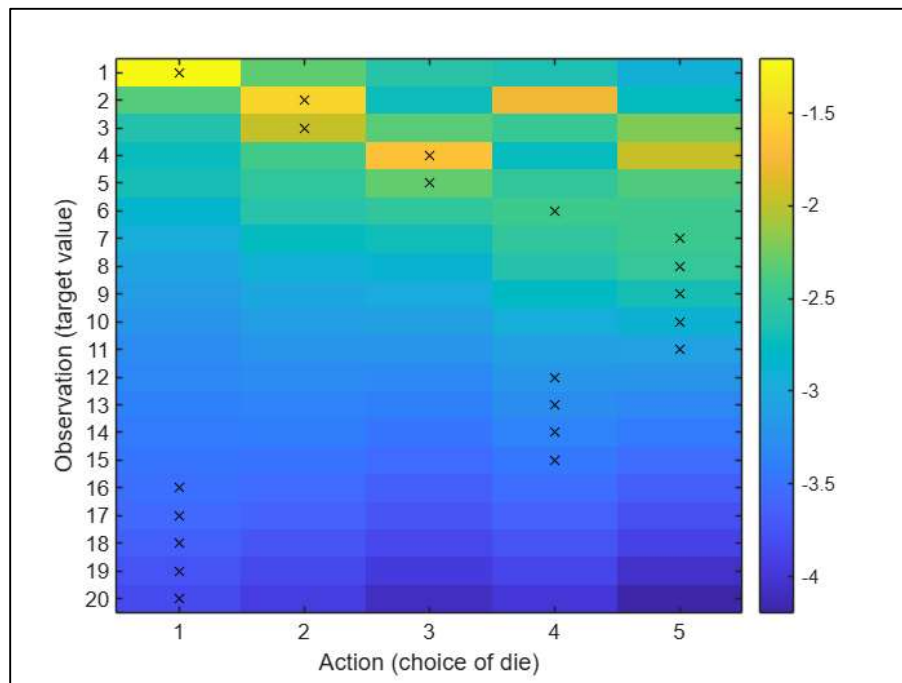


Figure 11: Learned Q-values and greedy policy of the trained DQN agent, showing the same qualitative policy structure as the Q-table agent.

| Metric               | rIQAgent (Q-table) | rIQNAgent (neural critic) |
|----------------------|--------------------|---------------------------|
| Episodes to stop     | 886 / 2000         | 425 / 2000                |
| Training duration    | 00:03:28           | 00:02:18                  |
| Final average reward | -2.99              | -3                        |

|                      |         |         |
|----------------------|---------|---------|
| Final episode reward | -4      | -2      |
| Final Episode Q0     | -4.3365 | -2.3017 |

### 3.4 Analysis Questions

#### Q6. What does the learning curve represent?

The learning curve is a plot of the reward obtained in each episode against the episode number, together with a moving average of that reward and the critic's initial-state value estimate Q0. It therefore represents the evolution of the quality of the agent's policy over the course of training: each point summarises one complete interaction between agent and environment, and the trend across points shows whether successive updates to the value estimates are actually producing better behaviour.

#### Q7. Why is the reward generally low during the initial training episodes?

At the start the Q-table holds no useful information and the  $\epsilon$ -greedy exploration rate is at its maximum, so dice are chosen essentially at random. The agent therefore overshoots or undershoots the target repeatedly, needs many rolls, and accumulates a large negative reward because each roll costs -1. In this run the effect is clearly visible: several episodes in the first 50 fell below -20, and the moving average actually declined from -4 to about -6.4 before recovering, as the initial value estimates were corrected downwards towards their true values.

#### Q8. What indicates that the agent is learning?

The clearest indicator is a sustained upward trend in the moving average of the episode reward rather than in any single episode — here, the rise from -6.4 near episode 40 to -2.99 at episode 886. Supporting indicators are the marked narrowing of the scatter of individual episode rewards (worst case improving from below -20 to about -8), Episode Q0 converging towards the observed returns, and, at the behavioural level, the greedy policy plot showing a clear structured mapping from target value to die rather than an arbitrary one.

#### Q9. Does the reward increase consistently throughout training? Explain.

No. The overall trend is upward but the curve is noisy throughout, and in the first 40 episodes the moving average actually falls before it rises. Several factors are responsible. The environment is stochastic, so even an optimal policy occasionally needs extra rolls because of unlucky dice.  $\epsilon$ -greedy exploration deliberately injects random actions. Q-learning updates only one table entry per step, so improvement is gradual and uneven across states. In the DQN run there is additional noise from random mini-batch sampling and from bootstrapping off a slowly updated target network. Consistency must therefore be judged from the moving average, not from individual episodes.

#### Q10. How can you identify convergence from the learning curve?

Convergence is indicated when the moving average becomes essentially flat over a substantial number of episodes, when the scatter of individual episodes settles into a stable band around that average, and when Episode Q0 also flattens and comes into reasonable agreement with the achieved returns. In this run the average reward stopped improving after roughly episode 350 and thereafter only oscillated between about -3.0 and -3.9. Because StopTrainingCriteria was set to "AverageReward", training also halted automatically — at episode 886, with the final result

reported as “Training finished after all agents reached stop training criteria” — which is itself direct evidence of convergence.

**Q11. What could cause fluctuations in the learning curve?**

Randomness in the environment (the dice outcome), continued  $\epsilon$ -greedy exploration, the fact that only the visited state–action pair is updated on each step, a learning rate large enough to overshoot, and a short averaging window that fails to smooth the noise. For the DQN agent there are two further sources: variance from random mini-batch sampling out of the replay buffer, and the non-stationary regression target created by the target network. Sudden non-recoverable drops, as opposed to scatter, would instead suggest an unstable learning rate or catastrophic forgetting.

**Q12. What happens if the agent is trained for a larger number of episodes?**

Once the policy has converged and  $\epsilon$  has decayed to its minimum, further episodes yield diminishing returns: the average reward stays flat while training time continues to grow. That is exactly why the run here stopped at 886 rather than continuing to the 2000-episode limit — the remaining 1114 episodes would have cost time without improving the policy. Extra training is not always harmless either: continued updates on a narrow, highly correlated set of experiences can destabilise a neural critic and cause the average reward to drift down. Conversely, in a problem that has not genuinely converged, further training can allow the agent to escape a locally optimal policy, which is why the stopping decision should be taken from the curve rather than fixed in advance.

**Q13. Why is it important to analyse the learning curve rather than considering only the final reward?**

The final reward is a single number and can be misleading. In this run the last episode scored  $-4$ , which is worse than the moving average of  $-2.99$  — judged on that number alone the agent would look poorer than it actually is, simply because of one unlucky sequence of dice. The curve reveals what a single number cannot: whether improvement was genuine and sustained, how many episodes were needed (sample efficiency), whether the agent had truly converged or was still improving, whether learning was stable or oscillatory, and whether exploration was adequate. It also exposes failure modes such as an early dip that never recovers or a peak followed by divergence, and it is the main diagnostic used when tuning the learning rate, exploration schedule and network size.

## **5. Conclusion**

The MATLAB Reinforcement Learning Onramp was completed in full, including all interactive exercises, MATLAB examples and chapter assessments. The complete RL workflow was carried out in MATLAB: specifying observation and action spaces, building the targetDice environment from step and reset functions, constructing both a tabular Q-table agent and a neural-network DQN agent, configuring optimiser, agent and training options, training the agents and evaluating the resulting policies.

The rlQAgent run converged in 886 of a possible 2000 episodes in 3 minutes 28 seconds, improving the moving-average reward from a low of about  $-6.4$  to a final value of  $-2.99$ , which corresponds to reaching the target in roughly three rolls instead of six or more. The learning curve showed an initial

deterioration while the value estimates were being corrected, a steep improvement between episodes 150 and 350, and a flat, stable region thereafter that triggered the automatic stopping criterion. The DQN agent reached a comparable policy at episode 425. The exercise demonstrated in practice that RL performance must be judged from the trend of the averaged reward across training rather than from any single episode — a point made concrete here by a final episode reward of  $-4$  sitting well below the converged average of  $-2.99$ .

## 6. Submission Checklist

- MATLAB Reinforcement Learning Onramp certificate
- Screenshot of the completed Onramp course —
- Screenshot of the trained RL agent / training progress — Figures 6, 7 and 10
- Screenshot of the learning curve — Figure 9
- Completed observation tables — Sections 2.5, 3.2 and 3.3
- Answers to all analysis questions — Sections 2.7 and 3.4

□

## Appendix A — Supporting Output from the Onramp

### A.1 Q-Table Before and After Training

The Q-table of the untrained agent contains no useful structure; after training, the values decrease monotonically with the target value, reflecting the fact that a larger target requires more rolls.

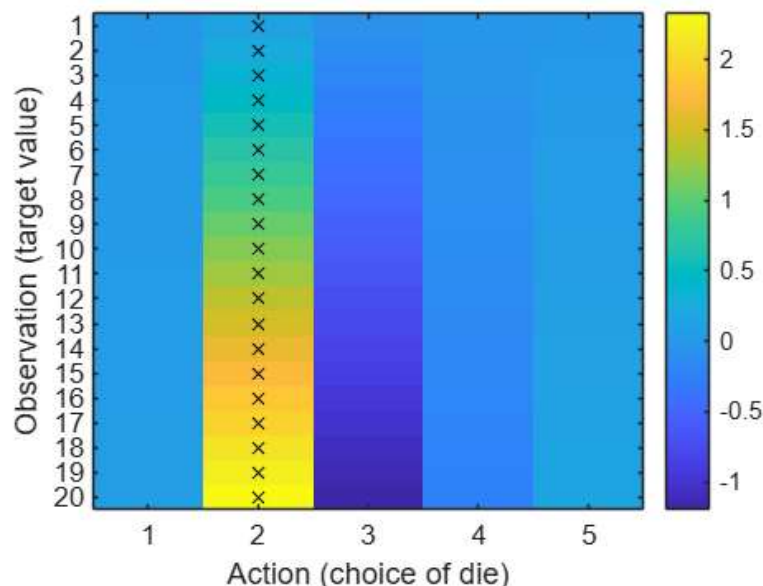


Figure A1: Q-values of the trained Q-table agent across all observations and actions.

Qvals = 20x5

|    | 1       | 2       | 3       | 4       | 5       |
|----|---------|---------|---------|---------|---------|
| 9  | -2.7251 | -2.7186 | -2.7377 | -2.7109 | -2.7109 |
| 10 | -2.8696 | -2.9018 | -2.7559 | -2.9198 | -2.9198 |
| 11 | -3.0708 | -3.0779 | -3.0881 | -3.0794 | -3.0794 |
| 12 | -2.9321 | -2.9127 | -2.9133 | -2.8819 | -2.8819 |
| 13 | -3.1540 | -3.1505 | -3.1574 | -3.1506 | -3.1506 |
| 14 | -3.2094 | -3.2082 | -3.2102 | -3.2084 | -3.2084 |
| 15 | -3.0539 | -3.0454 | -3.0520 | -3.0454 | -3.0454 |
| 16 | -2.9769 | -2.9834 | -2.9755 | -2.9690 | -2.9690 |
| 17 | -2.7860 | -2.7899 | -2.7937 | -2.7884 | -2.7884 |
| 18 | -2.6444 | -2.6427 | -2.6466 | -2.6479 | -2.6479 |
| 19 | -2.2520 | -2.2483 | -2.2553 | -2.2537 | -2.2537 |
| 20 | -1.8613 | -1.8704 | -1.8702 | -1.8718 | -1.8718 |

Figure A2: Numerical contents of the 21x5 Q-table after training.

## A.2 Critic Accuracy — Untrained vs. Trained

The value predicted by the critic should equal the negative of the average number of rolls needed to finish the game. Before training the prediction is flat and unrelated to the target; after training it follows the actual number of rolls closely.

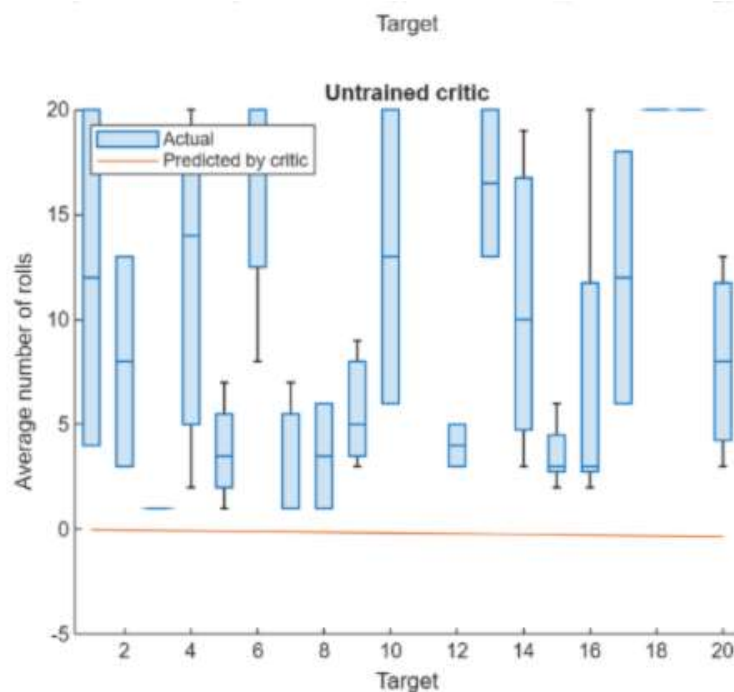


Figure A3: Untrained critic — predictions bear no relation to the actual number of rolls.

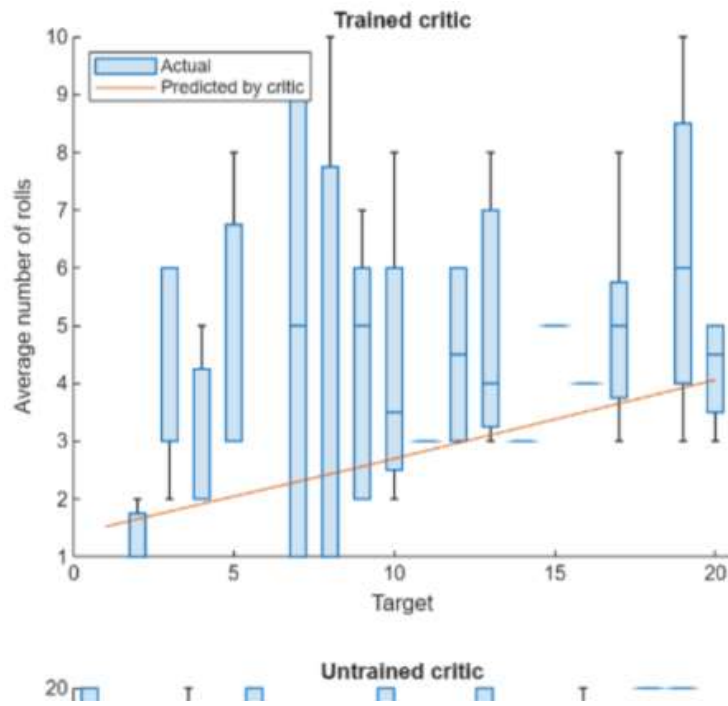


Figure A4: Trained critic — predicted value tracks the actual average number of rolls.

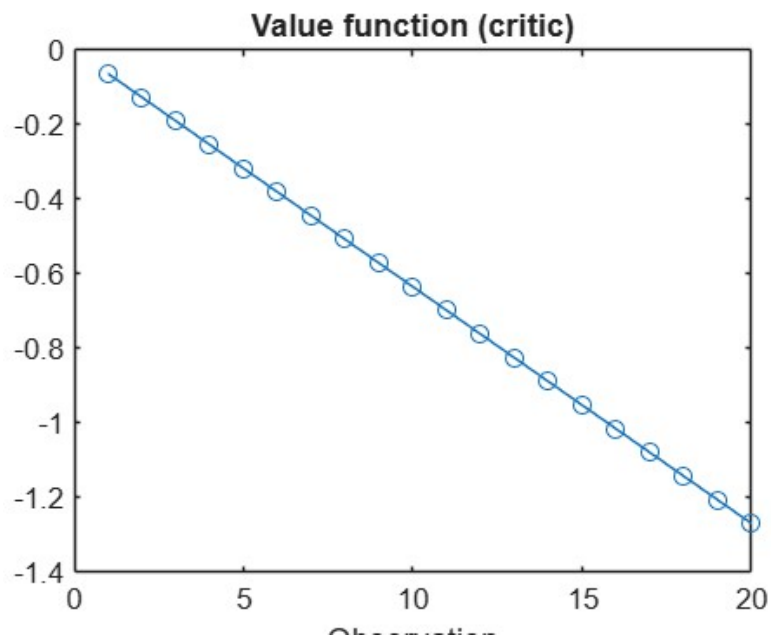


Figure A5: Value function learned by the PPO critic, decreasing approximately linearly with the target value.

### A.3 Warehouse Robot (Continuous Environment)

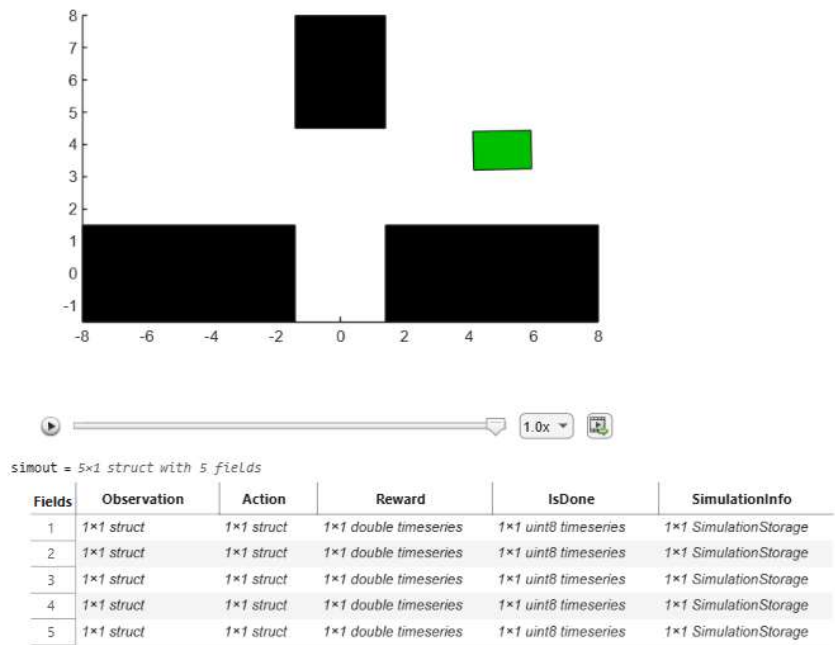


Figure A6: Simulation of the warehouse robot environment with the DDPG agent, and the resulting simout structure.

### A.4 MATLAB Console Output Captured

```
T =
  rlTable with properties:
    Table: [21x5 double]

% Greedy action (die) selected for each target value:
  1  2  2  3  1  4  5  4  4  3  5  4  2  5  2  4  1  2  2  1

agent =
  rldQNAgent with properties:
    ExperienceBuffer: [1x1 rl.replay.rlReplayMemory]
    AgentOptions: [1x1 rl.option.rldQNAgentOptions]
    UseExplorationPolicy: 0
    ObservationInfo: [1x1 rl.util.rlFiniteSetSpec]
    ActionInfo: [1x1 rl.util.rlFiniteSetSpec]
    SampleTime: 1
    UseGPUForLearning: 0

critic =
  rlVectorQValueFunction with properties:
    Normalization: "none"
    UseDevice: "cpu"
    Learnables: {6x1 cell}

criticopts =
  rlOptimizerOptions with properties:
    LearnRate: 1.0000e-03
    GradientThreshold: 1
    GradientThresholdMethod: "l2norm"
    L2RegularizationFactor: 1.0000e-04
    Algorithm: "adam"
```

```

agopts =
  rLDQNAgentOptions with properties:
      SampleTime: 1
      DiscountFactor: 1
      ExperienceBufferLength: 10000
      MiniBatchSize: 64
      NumWarmStartSteps: 64
      UseDoubleDQN: 1
      TargetSmoothFactor: 1.0000e-03
      TargetUpdateFrequency: 1

% Warehouse robot (continuous)
actor = rlContinuousDeterministicActor (Learnables: {6x1 cell})
critic = rlQValueFunction (Learnables: {8x1 cell})
alayers: input_1 (6 features) -> fc_1 (256) -> relu_body -> fc_body (256)
        -> body_output (ReLU) -> output (2) -> tanh

```

## A.5 Agent Selection Reference

Agents listed in increasing order of complexity; value-based (critic-only) and actor–critic agents are both available for each action-space type.

| Discrete Actions                                      | Continuous Actions                                      |
|-------------------------------------------------------|---------------------------------------------------------|
| Q-Learning — rlQAgent                                 | Deep Deterministic Policy Gradient (DDPG) — rIDDPGAgent |
| SARSA — rlSARSAAgent                                  | Twin-Delayed DDPG (TD3) — rITD3Agent                    |
| Deep Q-Network (DQN) — rLDQNAgent                     | Proximal Policy Optimization (PPO) — rlPPOAgent         |
| Proximal Policy Optimization (PPO) — rlPPOAgent       | Soft Actor-Critic (SAC) — rlSACAgent                    |
| Trust Region Policy Optimization (TRPO) — rlTRPOAgent | Trust Region Policy Optimization (TRPO) — rITRPOAgent   |

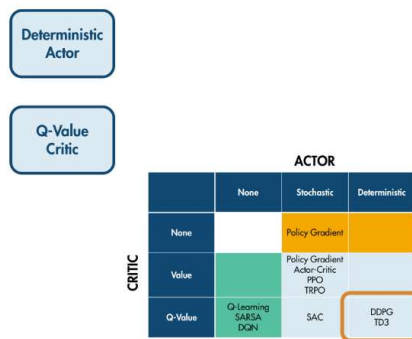


Figure A7: Actor–critic combination used by the DDPG agent — deterministic actor with a Q-value critic.